

Utilizing weather forecasts for power output prediction

Albert Juel Ross

ITU

Copenhagen, Denmark

alrj@itu.dk

I. INTRODUCTION

In 2020, the Orkney islands generated 128% of its electricity needs from renewables [1], where the bulk of this power generation came from wind turbines [2]. Orkney currently has limitations with its grid, as export is constrained to the capacity of two subsea cables [1]. For this reason, short-term forecasts of power generated from wind-energy could be desirable. The main focus of this report is building a reproducible machine learning pipeline that predicts total power output, given a forecast of wind speed and direction.

The data in question is a 90-day frame of wind speed and direction with another dataset showing the amount of power generated through wind turbines.

II. METHODOLOGY

Since ML and MLOps are such established fields, multiple frameworks and libraries exist in order to make the life of the engineer easier. All runtime dependencies are version-pinned [3]¹, which `python_env.yaml` installs into the isolated environment that `mlflow run` builds, so the pipeline reproduces from a clean checkout on another machine.

Most of the libraries are for convenience, however, two are worth specifically highlighting: `sklearn` [4], a very common ML library, and the usage of `mlflow` [5], a tool for comparing and versioning models.

The main discovery process has been in a python notebook, with models and model metrics being logged to `mlflow`. The process included building a meta-framework for automating experiments, and for building “immutable” pipelines². The main purpose of this meta-framework was to (1) make pipeline updates simple and immutable and (2) making experiments easy and exhaustive. As it can be hard to figure out what’s the best combination of transformers/regressors, the main method employed in this project is doing exhaustive search on pipelines. Exhaustive search in this instance means, building a pipeline with different trans-

formers, and trying all valid³ combinations. Each of these combinations is then tried on multiple regressors, thus all possible combinations of transformers and regressors are tried exhaustively. Each of these runs is recorded to `mlflow`, where the properties of individual runs can be inspected. For an example of the exhaustive pipelines see Appendix A.

Both feature transformations/engineering and the model itself are placed in a single pipeline. This means that model fitting and predictions are reproducible and simple. The only step which is not in the pipeline is the data-re-sampling⁴.

III. DATA ALIGNMENT

One of the main issues with the two datasets is the different cardinality and resolution. This is because the power dataset is sampled every minute, while the wind dataset is sampled every three hours. There are multiple ways of addressing this; either downsample the power dataset to match the 3 hour interval, upsample the wind dataset to match the minute interval, or do something in-between. The main benefit of downsampling is that the data is matched to the least common denominator, i.e. no data is created and no assumptions are made about its shape. This, however, comes at the cost of the amount of training data as well as the granularity of the predictions. Upsampling is the opposite of this: assumptions about the data’s shape have to be made, however, more data is gained and predictions can be made with higher granularity⁵.

I mainly tried 3 different approaches to data-re-sampling: (1) Downsample the power dataset to 3h intervals, (2) Resample both to 1h intervals, and (3) Upsampling the wind-dataset to 1m intervals. The approach which resulted in the best models based on the R^2 scores, was the 1 hour re-sampling. The main theory behind this is that the re-sampling does not introduce any signals. Thus if upsampling the wind dataset to minute granularity, then the model is not getting any actual signal. When downsampling the power-dataset to

¹The `uv` package manager was actually used for development, but `MLflow` does not seem to support this in the `MLproject` file, thus a separate requirements file exists.

²The builder returns a new pipeline on every call rather than mutating in place.

³Some pipeline steps are not compatible, e.g. providing both direction encoded as `sin/cos` and using the `OneHotEncoder` does not make much sense

⁴The `sklearn.Pipeline` is not made for re-sampling in general, for this the `imblearn.Pipeline` could be used, however I consider this out of scope for the project.

⁵Granularity here being the precision of the prediction, if the model has been trained on hourly data, it will be hard to predict what will happen the next second.

3 hours, then the actual power output just looks like noise. For this reason a good middle ground is doing both. There might be smooth changes between wind speed/direction which can be inferred on a 1 hour level, and power patterns might be well represented as an aggregate over this frame as-well. Thus the approach that best balances dataset distributions is to re-sample to 1-hour intervals. Upsampling is done by interpolating between actual data points, assuming wind speed changes smoothly, while direction is interpolated via forward filling. Downsampling is done by grouping the power dataset by hour and taking the mean of total power output.

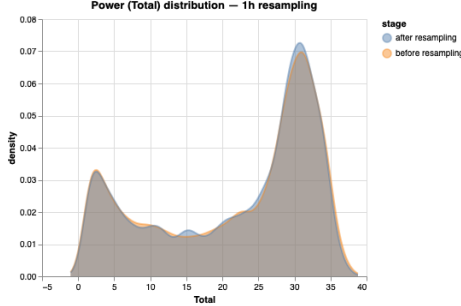


Fig. 1. Total power output distribution (Gaussian KDE) before and after down-sampling to a 1-hour interval (Mean over 1-hour frame)

As can be seen, this down-sampling causes an increase in the 30 MW range and in the 15 MW range. The probable reason for this is that averaging 60 samples per hour centers the distribution. This change results in a small reduction in standard deviation, specifically from $\sigma^2 \approx 11.148$ to $\sigma^2 \approx 11.019$.

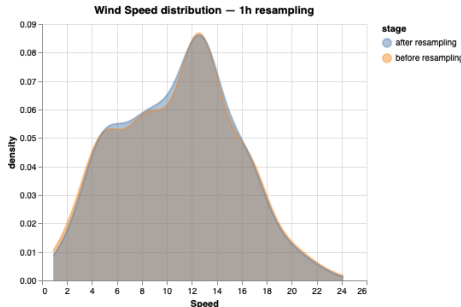


Fig. 2. Wind speed distribution (Gaussian KDE) before and after re-sampling to a 1-hour interval (Linear interpolation)

The wind speed seems minimally affected by the up-sampling, only causing some smoothing in the distribution; this makes sense as the interpolation is linear. This means that we gain resolution without sacrificing much of the datasets variance, specifically standard deviation drops by a small margin from $\sigma^2 \approx 4.745$ to $\sigma^2 \approx 4.623$.

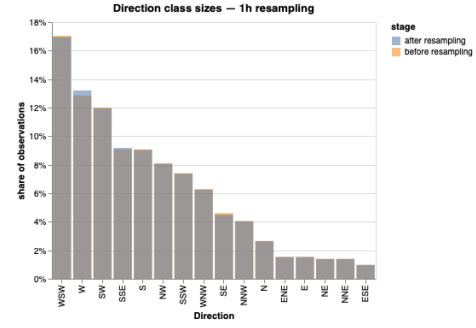


Fig. 3. Direction class shares before and after re-sampling to a 1-hour interval (forward fill)

Directions are forward filled, thus there is almost no difference between the raw dataset and the upsampled one. The main reason for the small difference is that there exist two gaps which are larger than 3 hours.

Measurement gaps	count
3h	713
6h	1
12h	1

This causes some over-representation of certain classes, but I choose to ignore it⁶.

IV. DATA PREPROCESSING

Due to the main methodology of doing exhaustive pipeline runs, multiple approaches have been taken, which may or may not end up in the optimal pipeline. This section will describe the approaches taken.

A. Train/test split

The data-splitting was done using Cross-Validation (CV). When using CV the number of folds chosen was 5. The main reason for this is that this is what TimeSeriesSplit defaults to. Other splits were tested, such as ShuffleSplit and GroupShuffleSplit, however, model performance got suspiciously good, suggesting data leakage. Using a time series split is also representative of the real world usage of the model, i.e. previous data points are used to predict future data. One issue with doing interpolation in combination with CV is that the split might land on interpolated features; since these are auto-correlated, it means that the model was actually trained on data which is in the test set. To fix this, splits are always performed on real samples, omitting any data which goes across the train test boundary. To further complicate the matter, I only want to test on real observations, which means that the real observations have to be found retroactively when computing the test score. This is done with a flag, and a column containing the real value and not the aggregate.

⁶The over-representation is minimal, so I conjecture that this problem is not worth chasing.

B. Missing values

The raw datasets contain no null values. Nulls are only introduced in the join and resampling step described in Section III; these are resolved at resampling time, such that they're not propagated. The null handling is done in the Speed column by linear interpolation, and Direction is handled by forward filling. Thus no imputation step is needed to fill nulls for input data values. There exists a rolling mean step in the pipeline, which introduces nulls in the first 11 rows of the dataframe, since the window has not yet been filled; these are imputed with the column mean.

C. Wind direction encoding

Two methods of direction encoding have been employed and tested. The first method is using a OneHotEncoder, which encodes categorical data by pivoting the categories to binary columns. The other method is encoding the directions to degrees and extracting sin and cos from those degrees. Since the compass is split into 16 classes, there are $\frac{360}{16} = 22.5^\circ$ between neighbouring classes. That is, if the classes are mapped to integers $i \in \{1, 2, \dots, 16\}$ in compass order starting at north, then the wind direction is computed as $\theta_i = i \cdot 22.5^\circ$, and the encoder emits the pair $(\sin \theta_i, \cos \theta_i)$. This also preserves the circular dependency which is discarded by the OneHotEncoder.

D. Feature scaling

With some models it can make sense to scale features as larger features might skew the loss function, causing other features to be undermined. To do this a StandardScaler was applied. A MinMaxScaler was also tried. The main difference between the StandardScaler and the MinMaxScaler is that the standard scaler removes the mean such that the dataset has unit variance [6] (i.e. the z-score is calculated) $z = \frac{x-\mu}{\sigma}$. The MinMaxScaler scales each feature such that it is in the range between 0 and 1 [7]. The best model ended up including neither scaler, but they're explained in this section for the sake of completeness and transparency. Another important note is that for boosted trees, scalars do not make any performance difference. The reason is that boosted trees make a series of "splits", meaning that monotonic transformations (such as scaling) have no real impact.

V. MODEL TRAINING AND EVALUATION

A. Models

Four models were evaluated against each other during the exhaustive runs.

- A linear regression model, LinearRegression
- A tree based model, HistGradientBoostingRegressor
- A support vector machine based model, SVR
- A multi layer perceptron model, MLPRegressor

The reasoning behind the different model choices is to test specific model classes against each other, to get a good baseline. For this reason the linear model was chosen for a linear baseline, the boosted trees for an ensemble based model, support vector for a kernel baseline and a MLP for a neural baseline. Had the data been upsampled to minute

granularity, SVR would have been painfully slow; LinearSVR would be the practical substitute in that case.

B. Evaluation metrics

The main metric used to evaluate these models is the mean R^2 score of 5 CV folds. Other supporting metrics are mean RMSE and mean MAE, as well as the best and worst of each. R^2 measures the proportion of variance which is explained by the model. A model with an R^2 of 0 is no better than just predicting the mean, 1 is a perfect fit, and a negative value means the model is worse than just predicting the mean. The main benefit of using R^2 is that maximizing R^2 equates to minimizing the sum of squared residuals. The metric does not, however, preserve any information about the unit.

VI. EXPERIMENT TRACKING WITH MLFLOW

MLflow was used to compare the models. The main way this was set up is that if the same data went into the model, and the model evaluation is the same, then it is the same experiment. Each run is then a variation of the different pipelines. To view how this was done in practice see Appendix B. As explained in Section V.B, the main metric used is mean R^2 . To find the model with the best mean R^2 I just sort by descending for that column in the run table. The UI also allows the data to be viewed as a histogram, however, because of the exhaustive runs, this was impractical. The project has autolog enabled, and utilizes manual metric/model/dataset/artifact logging. This is all done in the experiment builder. Some logging can be disabled to improve execution speed. The metrics logged are the ones presented in Section V.B.

VII. RESULTS

The best performing model was the SVR model utilizing the OneHotEncoder for direction encoding. Notably, the circular sin/cos encoding described in Section IV.C did not win, despite preserving more structure. A plausible explanation is that SVR already handles the one-hot columns well, so an explicitly circular representation buys little on this dataset. Furthermore, scaling seems to have a negative impact on the model, even though a support vector kernel usually benefits from feature scaling. There could be many reasons for this, but it is out of scope for this report. This model managed a mean R^2 score of 0.70. The best fold for this model was fold 4⁷ managing an R^2 score of ~ 0.83 , while the worst was fold 2 at ~ 0.55 . An interesting observation is that the different models seem to perform worse on the same fold, specifically fold 2 seems to contain some data which lowers the ceiling considerably: no pipeline out of the 144 scores above 0.63 on it, the lowest maximum of any fold.

⁷Fold 4 is the last fold due to zero indexing

TABLE II
PER-FOLD R^2 STATISTICS ACROSS ALL 144 RUNS.

	Fold 0	Fold 1	Fold 2	Fold 3	Fold 4
Mean	-3.76e22	0.6229	0.3518	0.6040	0.7963
Median	0.6134	0.6635	0.5135	0.6185	0.8081
Min	-3.09e24	0.2864	-0.3220	0.4418	0.6631
Max	0.7411	0.7802	0.6283	0.6532	0.8491

If fold 2 is excluded, the mean R^2 of the best model rises from 0.70 to 0.74, a gain of 3.8 *pp*. This is probably explained by the fact that there are a lot of missing observations in the power dataset.

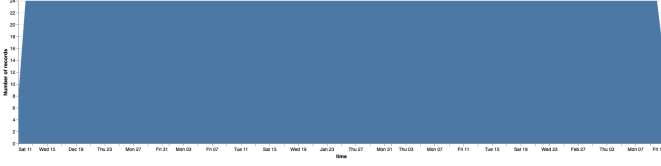


Fig. 4. Upsampled wind data points

As can be seen in the resampled wind dataset, there are 24 observations per day.

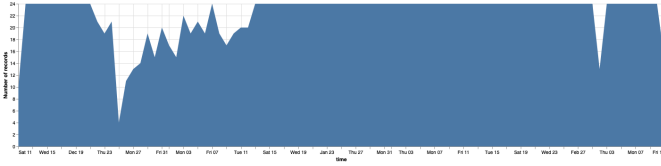


Fig. 5. Downsampled power data point

However, for the power data it can be seen that for some days there are missing data points, i.e. days with fewer than 24 hourly observations. Coincidentally, the third fold (fold 2) ends at 2022-01-28 05:00:00, which is just two weeks after the data gap, meaning there is quite a large data gap in the range of this fold. This could be the explanation to the unusually terrible performance.

A. Future predictions

The best model was tested against future.csv. For this the model was evaluated with a single unshuffled train/test split rather than CV, since the folds were only needed for model comparison. The model that is logged and later served is then refitted on the full dataset, so that the forecast is produced by a model which has seen every available observation. The output was appended (red line) to the existing dataset (blue line) and plotted, see Figure 6.

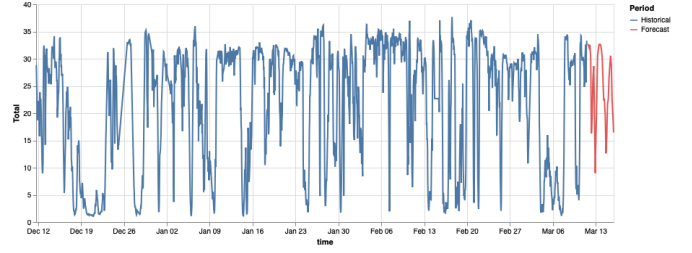


Fig. 6. Forecast for future total power output (MW) based on weather forecast data

VIII. MODEL SERVING

The best model has been registered to the MLflow registry, see Appendix C. To serve it the following command was used:

```
MLFLOW_TRACKING_URI=http://127.0.0.1:5000 mlflow
models serve -m "models:/SVR power output prediction/3" -p
5001 --env-manager local
```

The console output from this command can be seen in Appendix D. The curl and the model response can be seen in Appendix E.

IX. REFLECTION

A. Training data window size

The window size is only 90 days; the hourly grid is therefore 2158 rows ($\sim 90 \times 24$), and after the inner join with the power dataset 2008 rows remain, of which only 659 are real wind observations. With a 5 fold split, this means that each fold adds only a small amount of data. This could cause the model to not have enough training data to properly generalize. This claim could be supported by the fact that from fold 2 the model performance increases with more data. However, I consider this claim to be on shaky ground at best. Another point is that there could be seasonal patterns which would not be shown in the data, since it only contains a subset of the year. As it stands, including time-derived features in the training data seems to hurt the model more than it helps. If there were two years of training data there could be a hidden signal in the time and day of the year, but there is no way to be sure.

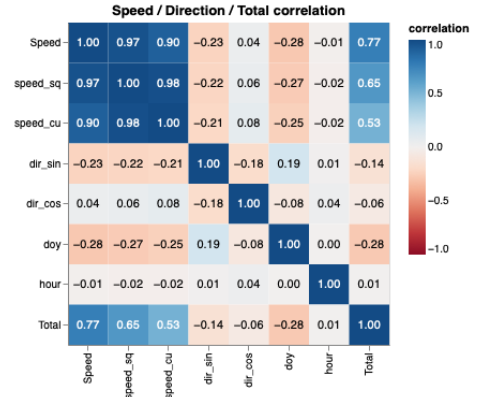


Fig. 7. Correlation matrix

The correlation matrix shows that there is a small negative correlation between the ordinal day (doy) and the total power output, and the same correlation for speed. This suggests that doym is largely a proxy for wind speed rather than an independent signal, which is consistent with the observation that adding time-derived features hurt performance.

B. Limitations and improvements

As already mentioned in the section above, there might be training data limitations, due to the small size of the data. Other limitations regarding the dataset could be that the distribution of wind speed is not uniform, meaning that there are far more observations within a specific range. This might bias the model, causing it to be more uncertain for wind speeds outside that range. Within the scope of dataset imbalance, there is also the imbalance referenced in Section III (see Figure 3). The wind directions are not evenly distributed which could be another limiting factor in regards to model performance.

The way in which the optimal transformer/regressor combination was found was through exhaustive combinations of these, see Section II. This is a valid way to do model comparisons, but the main limiting factor is that the current approach does not test different hyperparameter configurations. In addition, testing different models through exhaustive iterations is not the fastest approach to find a good model pipeline; for this, something like GridSearchCV could have been used. GridSearchCV also has support for hyperparameter tuning given a discrete set of values. However, a dedicated hyperparameter tuning framework like optuna [8], [9] would be preferred.

X. USE OF GENERATIVE AI

Anthropic’s Claude [10] was used during this project as a coding assistant and a sounding board. Specifically it helped refactor certain parts of the code and was used as a debugging tool. Other parts of the code are completely generated; this is the code for the plots seen in this report. Claude also participated in the writing of this report for scaffolding section headers from the README, as a reviewer, and for generating bibliography references into the bibtex format. However, none of the section contents were written by AI.

A. Example of code refactor

One of the code refactors done by Claude concerns the experiment builder, which was originally also responsible for building the pipeline. After building the ColumnTransformerBuilder, I realized that this architecture was not sound, so Claude refactored the existing building logic into the PipelineBuilder.

B. Example of code generation

The code for the figures and other plotting was built by Claude.

APPENDIX A

Given the following experiment:

```
ExperimentBuilder()
.with_transformer(DirectionEncoder())
.with_transformer(OneHotEncoder())
.with_transformer(StandardScaler())
.using_regressors(MLPRegressor(), SVR())
```

The following combinations are tried and logged:

```
pipeline_1 = Pipeline([
    MLPRegressor()
])
pipeline_2 = Pipeline([
    SVR()
])
pipeline_3 = Pipeline([
    DirectionEncoder(),
    MLPRegressor()
])
pipeline_4 = Pipeline([
    DirectionEncoder(),
    SVR()
])
pipeline_5 = Pipeline([
    OneHotEncoder(),
    MLPRegressor()
])
pipeline_6 = Pipeline([
    OneHotEncoder(),
    SVR()
])
pipeline_7 = Pipeline([
    StandardScaler(),
    MLPRegressor()
])
pipeline_8 = Pipeline([
    StandardScaler(),
    SVR()
])
pipeline_9 = Pipeline([
    DirectionEncoder(),
    StandardScaler(),
    MLPRegressor()
])
# And so on
```

APPENDIX B

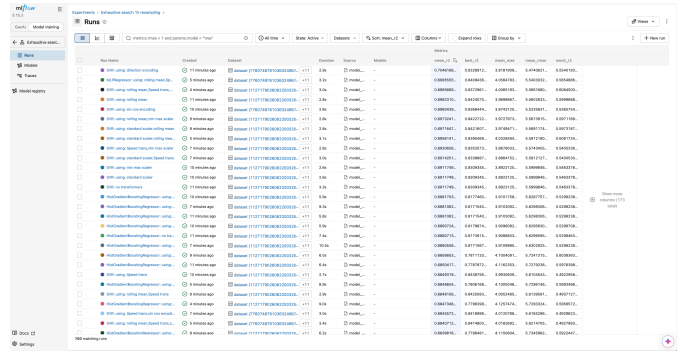


Fig. 8. Screenshot from an experiment done in MLflow

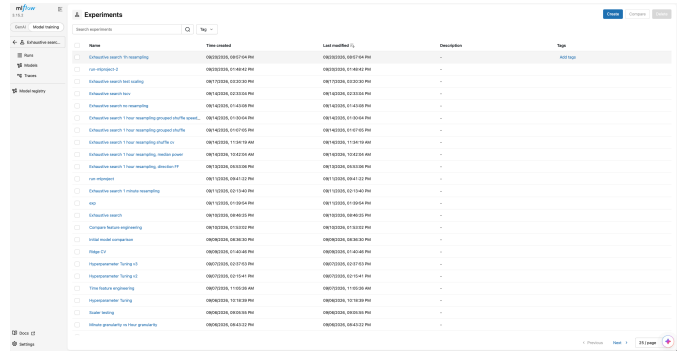


Fig. 9. Screenshot of multiple different experiments in MLflow

APPENDIX C

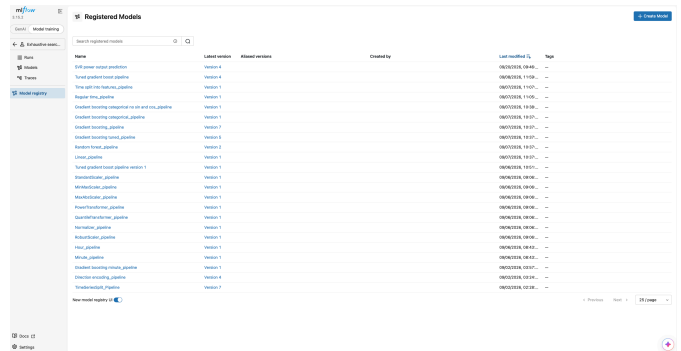


Fig. 10. Screenshot of the mlflow model registry

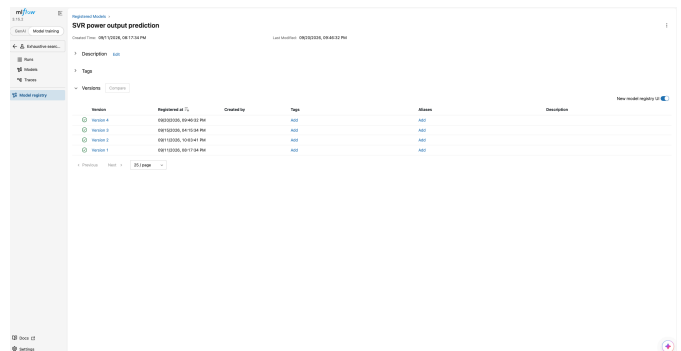


Fig. 11. Screenshot SVR model and versions

APPENDIX D

```
(bdm-assignment-mlflow) (base) → BDM_Assignment_MLflow git:(main) × MLFLOW_TRACE=1 mlflow models serve -m "models:SVR power output prediction/3" -p 5001 --env-manager local
Downloading artifacts: 100%|██████████| 5/5 [00:00<00:00, 341.46it/s]
2026/09/16 14:20:52 INFO mlflow.models.flavor_backend_registry: Selected backend for flavor 'python_function'
Downloading artifacts: 100%|██████████| 5/5 [00:00<00:00, 461.43it/s]
2026/09/16 14:20:52 INFO mlflow.pyfunc.backend: === Running command 'exec uvicorn n --host 127.0.0.1 --port 5001 --workers 1 mlflow.pyfunc.scoring_server.app:app'
INFO: Started server process [93687]
INFO: Waiting for application startup.
INFO: Application startup complete.
INFO: Uvicorn running on http://127.0.0.1:5001 (Press CTRL+C to quit)
INFO: 127.0.0.1:60374 - "POST /invocations HTTP/1.1" 200 OK
```

Fig. 12. Screenshot from terminal serving the model

APPENDIX E

```
(base) [?] report git:(main) × curl -X POST
http://127.0.0.1:5001/invocations \
-H "Content-Type: application/json" \
-d '{"dataframe_split": {"columns": ["Speed", "Direction"],
"data": [[15.2, "SE"], [16.09, "SE"], [16.99, "SE"], [16.09, "SSE"], [12.96, "SSE"], [8.05, "SSE"], [8.94, "SSE"], [11.18, "SE"], [12.07, "SSE"], [8.94, "SSW"], [5.81, "S"], [8.05, "SSE"], [9.83, "SE"], [12.96, "SE"], [13.86, "SE"], [15.2, "SE"], [15.2, "SE"], [15.2, "SSE"], [13.86, "SSE"], [12.96, "SSE"], [11.18, "S"], [9.83, "S"], [9.83, "SSW"], [8.94, "SSW"], [7.15, "SSW"], [8.05, "SW"], [9.83, "SW"], [9.83, "SSW"], [11.18, "S"], [12.07, "S"], [12.96, "S"], [12.07, "S"], [9.83, "S"], [8.94, "SSW"], [8.05, "SSW"]]]}}'
{"predictions": [
32.37608279215685,
32.29420614801475,
31.712189816285054,
32.52496851750467,
30.47796517171369,
16.153040706802685,
19.264870107670728,
26.111244935706388,
28.682635673045645,
19.12448041465684,
8.625975635378996,
16.153040706802685,
21.97313224809883,
30.188962449474506,
31.485520334428543,
32.37608279215685,
32.37608279215685,
32.625706661538295,
31.760403226975388,
30.47796517171369,
26.203249351412914,
22.076228921334746,
22.1474462697079,
19.12448041465684,
12.41288167941098,
15.495405748553758,
21.60639612293425,
```

```
22.1474462697079,
26.203249351412914,
28.466332934848715,
30.264418023709005,
28.466332934848715,
22.076228921334746,
19.12448041465684,
16.01400943604054
```

```
}
}
```

APPENDIX F

The full source code for this project is available at [11]:
https://github.com/AlbertRossJoh/BDM_Assignment_MLflow.

REFERENCES

- [1] Islands Centre for Net Zero, “Orkney — Island Profile.” [Online]. Available: <https://www.icnz.org/island-profiles/orkney>
- [2] VisitOrkney, “Renewables — Energy in Orkney.” [Online]. Available: <https://www.orkney.com/life/energy/renewables>
- [3] A. J. Ross, “requirements.txt — pinned runtime dependencies.” [Online]. Available: https://github.com/AlbertRossJoh/BDM_Assignment_MLflow/blob/main/requirements.txt
- [4] F. Pedregosa *et al.*, “Scikit-learn: Machine Learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
- [5] M. A. Zaharia *et al.*, “Accelerating the Machine Learning Lifecycle with MLflow,” *IEEE Data Eng. Bull.*, vol. 41, pp. 39–45, 2018, [Online]. Available: <https://api.semanticscholar.org/CorpusID:83459546>
- [6] scikit-learn developers, “StandardScaler.” [Online]. Available: <https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.StandardScaler.html>
- [7] scikit-learn developers, “MinMaxScaler.” [Online]. Available: <https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.MinMaxScaler.html#sklearn.preprocessing.MinMaxScaler>
- [8] T. Akiba, S. Sano, T. Yanase, T. Ohta, and M. Koyama, “Optuna: A Next-Generation Hyperparameter Optimization Framework,” in *The 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, 2019, pp. 2623–2631.
- [9] Optuna developers, “Optuna Documentation.” [Online]. Available: <https://optuna.readthedocs.io/en/stable/>
- [10] Anthropic, “Claude (Sonnet).” [Online]. Available: <https://www.anthropic.com/claude>
- [11] A. J. Ross, “Wind power forecasting pipeline — source repository.” [Online]. Available: https://github.com/AlbertRossJoh/BDM_Assignment_MLflow